

Selection Sort

```

Algorithm SelectionSort(array)
  n = length(array)
  for i from 0 to n - 1:
    minIndex = i
    for j from i + 1 to n - 1:
      if array[j] < array[minIndex]:
        minIndex = j
    swap(array[i], array[minIndex])
end Algorithm

```

```

def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

array = [64, 25, 12, 22, 11]
sorted_array = selection_sort(array)
print(sorted_array)

[11, 12, 22, 25, 64]

```

Analisa Kompleksitas: Waktu (Best/Avg/Worst) : $O(n^2)$. Ia selalu memindai seluruh sisa array terlepas dari apakah data sudah terurut atau belum.

Ruang: $O(1)$ (In-place).

Insertion Sort

```

Algorithm InsertionSort(array)
  for i from 1 to length(array) - 1:
    key = array[i]
    j = i - 1
    while j >= 0 and array[j] > key:
      array[j + 1] = array[j]
      j = j - 1
    array[j + 1] = key
end Algorithm

```

```

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

array = [64, 25, 12, 22, 11]
sorted_array = insertion_sort(array)
print(sorted_array)

```

```
[11, 12, 22, 25, 64]
```

Waktu: Best Case $O(n)$ (jika data sudah terurut), Average/Worst Case $O(n^2)$. Ruang: $O(1)$.

Mergesort

```

Algorithm MergeSort(array)
  if length(array) <= 1:

```

```

        return array

    mid = length(array) / 2
    left = MergeSort(left half of array)
    right = MergeSort(right half of array)

    return Merge(left, right)
end Algorithm

Algorithm Merge(left, right)
    create empty result list
    while left and right are not empty:
        if left[0] <= right[0]:
            append left[0] to result
            remove left[0]
        else:
            append right[0] to result
            remove right[0]
    append remaining elements in left or right to result
    return result
end Algorithm

```

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i]); i += 1

```

```
        else:
            result.append(right[j]); j += 1
        result.extend(left[i:])
        result.extend(right[j:])
        return result

array = [64, 25, 12, 22, 11]
sorted_array = merge_sort(array)
print(sorted_array)
```

```
[11, 12, 22, 25, 64]
```

Waktu: $O(n \log n)$ stabil untuk semua kondisi (Best, Avg, Worst).

Ruang: $O(n)$ karena membutuhkan array tambahan untuk proses penggabungan.

Quicksort

```
Algorithm QuickSort(array, low, high)
    if low < high:
        pivotIndex = Partition(array, low, high)
        QuickSort(array, low, pivotIndex - 1)
        QuickSort(array, pivotIndex + 1, high)
    end Algorithm

Algorithm Partition(array, low, high)
    pivot = array[high]
    i = low - 1
    for j from low to high - 1:
        if array[j] <= pivot:
            i = i + 1
            swap(array[i], array[j])
    swap(array[i + 1], array[high])
    return i + 1
end Algorithm
```

```
def quicksort(arr):  
    if len(arr) <= 1:  
        return arr  
    pivot = arr[len(arr) // 2]  
    left = [x for x in arr if x < pivot]  
    middle = [x for x in arr if x == pivot]  
    right = [x for x in arr if x > pivot]  
    return quicksort(left) + middle + quicksort(right)  
  
array = [64, 25, 12, 22, 11]  
sorted_array = quicksort(array)  
print(sorted_array)
```

```
[11, 12, 22, 25, 64]
```

Waktu: Average/Best Case $O(n \log n)$. Worst Case $O(n^2)$ (terjadi jika pivot yang dipilih selalu elemen terkecil/terbesar, misal pada data yang sudah terurut). Ruang: $O(\log n)$ untuk tumpukan rekursi

